

ASSIGNMENT 1

Name: Om Upadhyay

Section: NA

Class Roll No.: 51

Q1. HTTP Request–Response Cycle

Explain the HTTP Request–Response Cycle with a suitable diagram. Describe the major steps involved when a client (browser) requests a resource from a web server and receives a response.

Ans.

The **HTTP Request–Response Cycle** is the process through which a client, usually a web browser, communicates with a web server to request and receive resources such as HTML pages, CSS files, JavaScript files, images, or data.

Major Steps Involved

1. User requests a resource

The process begins when a user enters a URL in the browser, clicks a link, or performs an action that requires a resource from a server. For example:

<https://example.com/index.html>

2. DNS Lookup

The browser needs the IP address of the server. It performs a **DNS (Domain Name System) lookup** to convert the domain name into an IP address.

example.com → 93.184.216.34

3. Connection is established

The browser establishes a network connection with the server. For HTTPS websites, a secure **TLS connection** is also established.

4. Browser sends an HTTP request

The browser sends an HTTP request containing information such as the HTTP method, requested resource, headers, and sometimes a request body.

Example:

GET /index.html HTTP/1.1

Host: example.com

Common HTTP methods include:

- **GET** – retrieve a resource
- **POST** – submit data

- **PUT** – update data
- **DELETE** – delete data

5. Server processes the request

The web server receives the request and determines what needs to be returned. It may retrieve a static file, execute server-side code, communicate with a database, or perform other processing.

6. Server sends an HTTP response

After processing the request, the server sends a response back to the browser.

Example:

HTTP/1.1 200 OK

Content-Type: text/html

The response usually contains:

- **Status code**
- **Response headers**
- **Response body**

Common status codes include:

Status Code	Meaning
200	OK / Request successful
301	Permanently redirected
404	Resource not found
500	Internal server error

7. Browser processes the response

The browser receives and interprets the response. If the response contains HTML, the browser parses it and may make additional HTTP requests for referenced resources such as CSS, JavaScript, images, and fonts.

8. Web page is rendered

Finally, the browser combines the received resources and renders the webpage on the screen for the user.

Q2. Introduction to Web Technologies

Briefly explain the purpose and key features of the following web technologies:

1. HTML
2. CSS
3. JavaScript
4. Bootstrap
5. jQuery
6. React.js

Ans.

Web technologies are tools and languages used to create, design, and develop websites and web applications. Each technology has a specific role in the development process.

1. HTML

HTML (HyperText Markup Language) is the standard markup language used to create the **structure and content of web pages**.

Purpose:

- Defines the structure of a webpage.
- Organizes content such as headings, paragraphs, images, links, tables, and forms.

Key Features:

- Uses tags and elements to structure content.
- Supports multimedia such as images, audio, and video.
- Provides forms for collecting user input.
- Supports semantic elements such as <header>, <nav>, <section>, and <footer>.
- Works with CSS and JavaScript to create complete web pages.

Example:

```
<h1>Welcome to My Website</h1>  
<p>This is a webpage.</p>
```

2. CSS

CSS (Cascading Style Sheets) is used to **style and design HTML webpages**.

Purpose:

- Controls the appearance and layout of webpages.
- Makes websites visually attractive and responsive.

Key Features:

- Controls colors, fonts, spacing, borders, and backgrounds.
- Provides layout systems such as **Flexbox and Grid**.
- Supports responsive web design using media queries.
- Provides animations and transitions.
- Allows styles to be reused across multiple webpages.

Example:

```
h1 {  
  color: blue;  
  font-size: 32px;  
}
```

3. JavaScript

JavaScript is a programming language used to add **interactivity and dynamic behavior** to webpages.

Purpose:

- Makes webpages interactive.
- Allows webpages to respond to user actions without requiring a complete page reload.

Key Features:

- Handles events such as clicks, keyboard input, and mouse movements.
- Can modify HTML and CSS dynamically through the DOM.
- Performs form validation.
- Supports asynchronous operations using techniques such as Fetch and Promises.
- Can communicate with APIs and web servers.
- Supports modern programming concepts such as classes, modules, and arrow functions.

Example:

```
function showMessage() {  
    alert("Hello, World!");  
}
```

4. Bootstrap

Bootstrap is a popular **front-end CSS framework** used to create responsive and mobile-friendly websites quickly.

Purpose:

- Provides ready-made CSS styles and UI components.
- Simplifies responsive webpage development.

Key Features:

- Responsive grid system.
- Pre-designed buttons, forms, cards, navigation bars, and modals.
- Mobile-first design.
- Utility classes for spacing, colors, alignment, and display.
- Can be customized according to project requirements.

Example:

```
<button class="btn btn-primary">Click Me</button>
```

5. jQuery

jQuery is a JavaScript library designed to simplify common JavaScript tasks.

Purpose:

- Makes DOM manipulation, event handling, animations, and AJAX operations easier.

Key Features:

- Simple syntax for selecting and modifying HTML elements.
- Easy event handling.
- Provides animation effects.
- Simplifies AJAX requests.
- Provides cross-browser compatibility support.
- Has a large collection of plugins.

Example:

```
$("#message").text("Hello World!");
```

Here, jQuery selects the element with the ID message and changes its text.

6. React.js

React.js is a JavaScript library used to build **interactive and dynamic user interfaces**, particularly for modern web applications.

Purpose:

- Helps developers create reusable UI components.
- Commonly used for developing single-page applications (SPAs).

Key Features:

- **Component-based architecture** for reusable UI components.
- Uses a **Virtual DOM** to efficiently update the interface.
- Supports **props** for passing data between components.
- Uses **state** to manage changing data.

- Supports event handling and conditional rendering.
- Large ecosystem and community support.
- Can be used with other libraries and frameworks to build complete web applications.

Example:

```
function Welcome() {  
  return <h1>Welcome to React!</h1>;  
}
```

Q3. Client-Side vs. Server-Side Technologies

What are client-side technologies and server-side technologies? Explain the difference between them with suitable examples.

Ans.

Client-Side Technologies

Client-side technologies are technologies that run directly in the user's web browser. They are mainly used to create the **appearance, structure, and interactive behavior** of a webpage.

Examples: HTML, CSS, JavaScript, React.js, Bootstrap.

Example: When a user clicks a button and the webpage changes without contacting the server, JavaScript can handle the action in the browser.

Server-Side Technologies

Server-side technologies run on the **web server**. They are used to process requests, perform business logic, interact with databases, authenticate users, and generate or provide data to the client.

Examples: Node.js, PHP, Python, Java, and Ruby.

Example: When a user logs into a website, the server can verify the username and password against a database and send the login result back to the browser.

Difference Between Client-Side and Server-Side Technologies

Client-Side	Server-Side
Runs in the user's browser.	Runs on the web server.
Handles webpage structure, styling, and interaction.	Handles business logic, databases, and authentication.

Code is generally sent to the browser.	Code generally remains on the server.
Examples: HTML, CSS, JavaScript, React.js.	Examples: Node.js, PHP, Python, Java.
Can provide immediate interaction without a server request for some operations.	Processes requests and sends responses to the client.

Q4. Block-Level Elements in HTML

What are block-level elements in HTML? Explain their characteristics and provide at least three examples.

Ans.

Block-level elements in HTML are elements that normally start on a **new line** and take up the **full available width** of their parent container.

Characteristics

- They start on a new line.
- They usually occupy the full width available.
- They can contain other elements, including block-level and inline elements, depending on the element.
- CSS properties such as **width**, **height**, **margin**, and **padding** can generally be applied to them.
- They are commonly used to structure and organize the layout of a webpage.

Examples

1. **<div>** – A general-purpose container used to group and structure content.
2. **<p>** – Defines a paragraph of text.
3. **<h1> to <h6>** – Define headings of different levels.
4. **<section>** – Defines a section of related content.
5. **<header>** – Defines the header section of a webpage.

Example

```
<div>
  <h1>My Website</h1>
  <p>This is a paragraph.</p>
</div>
```

Here, <div>, <h1>, and <p> are block-level elements, so each normally begins on a new line.

Q5. Semantic HTML Elements

What are semantic elements in HTML? Explain their importance in web development and provide suitable examples.

Ans.

Semantic HTML elements are HTML elements that clearly describe the **meaning and purpose of their content** to both the browser and the developer.

Importance in Web Development

- **Improves readability:** Makes HTML code easier for developers to understand and maintain.
- **Improves accessibility:** Helps screen readers and other assistive technologies understand the structure of a webpage.
- **Improves SEO:** Search engines can better understand the content and structure of a webpage.
- **Provides better structure:** Clearly separates different sections of a webpage.
- **Makes code more meaningful:** Developers can understand the purpose of an element without examining its content.

Examples of Semantic Elements

- <header> – Represents the introductory or header section of a webpage.
- <nav> – Defines navigation links.
- <main> – Contains the main content of the webpage.
- <section> – Represents a section of related content.
- <article> – Represents independent content such as a blog post or news article.
- <aside> – Contains related or supplementary content.
- <footer> – Represents the footer section of a webpage.

Example

```
<header>  
  <h1>My Website</h1>  
</header>
```

```
<nav>
```

```

        <a href="#">Home</a>
        <a href="#">About</a>
    </nav>

    <main>
        <section>
            <h2>About Us</h2>
            <p>Welcome to our website.</p>
        </section>
    </main>

    <footer>
        <p>© 2026 My Website</p>
    </footer>

```

These elements make the webpage structure clear and meaningful compared with using generic <div> elements for every section.

Q6. Creating a Table Using HTML

Write HTML code to create the following table:

Sr. No.	Name	Designation	Department
1	John	Manager	Sales
2	Smith	Sr. Manager	Marketing
3	Jane	Manager	HR

The table should include appropriate table headings, rows, and columns.

Ans.

```

<!DOCTYPE html>

<html>

    <head>

        <title>Employee Table</title>

    </head>

    <body>

```

```
<table border="1">

  <tr>

    <th>Sr. No.</th>

    <th>Name</th>

    <th>Designation</th>

    <th>Department</th>

  </tr>
```

```


  <tr>

    <td>1</td>

    <td>John</td>

    <td>Manager</td>

    <td>Sales</td>

  </tr>
```

```


  <tr>

    <td>2</td>

    <td>Smith</td>

    <td>Sr. Manager</td>

    <td>Marketing</td>

  </tr>
```

```


  <tr>

    <td>3</td>
```

```
<td>Jane</td>

<td>Manager</td>

<td>HR</td>

</tr>

</table>
```

```
</body>
```

```
</html>
```

Q7. Creating a Nested List Using HTML

Write HTML code to create the following list using an appropriate combination of **ordered/unordered lists**:

- River
 - Ganga
 - Yamuna
 - Brahmaputra
 - Narmada

Ans.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Nested List</title>
```

```
</head>
```

```
<body>
```

```
<ul>
```

```
<li>River
```

```
<ul>
```

```
<li>Ganga</li>

<li>Yamuna</li>

<li>Brahmaputra</li>

<li>Narmada</li>

</ul>

</li>

</ul>

</body>

</html>
```

Q8. Types of CSS

Explain the different types of CSS used for applying styles to an HTML document. Describe each type with a suitable example.

There are **three main types of CSS** used to apply styles to an HTML document:

1. Inline CSS

Inline CSS is applied directly to an HTML element using the `style` attribute.

Example:

```
<p style="color: blue; font-size: 20px;">
    This is a paragraph.
</p>
```

Here, the CSS is written directly inside the `<p>` element.

2. Internal CSS

Internal CSS is written inside a `<style>` tag within the `<head>` section of an HTML document. It is useful when styles are required for a single webpage.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <style>
    h1 {
      color: red;
      text-align: center;
    }
  </style>
</head>
<body>

  <h1>Welcome to My Website</h1>

</body>
</html>
```

Here, the styles are defined inside the `<style>` tag.

3. External CSS

External CSS is written in a separate `.css` file and linked to the HTML document using the `<link>` element. It is useful for applying the same styles to multiple webpages.

HTML file:

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <h1>Welcome to My Website</h1>

</body>
</html>
```

style.css:

```
h1 {  
    color: green;  
    text-align: center;  
}
```

Here, the CSS is stored in a separate `style.css` file.

Q9. Practical Understanding

Differentiate between HTML, CSS, and JavaScript based on their role in developing a web page. Explain what each technology contributes to a website.

Ans.

Technology	Role	What It Contributes
HTML	Structure	Creates the content and structure of the webpage, such as headings, paragraphs, images, links, and forms.
CSS	Presentation	Controls the appearance and layout, including colors, fonts, spacing, sizes, and responsiveness.
JavaScript	Behavior	Adds interactivity and dynamic functionality, such as button actions, form validation, animations, and updating content.

HTML

HTML (HyperText Markup Language) provides the basic structure of a webpage.

Example:

```
<h1>Welcome</h1>  
<p>This is my website.</p>
```

It defines **what content appears** on the webpage.

CSS

CSS (Cascading Style Sheets) is used to style and design the HTML content.

Example:

```
h1 {  
    color: blue;
```

```
    text-align: center;
}
```

It defines **how the content looks**.

JavaScript

JavaScript adds functionality and interactivity to a webpage.

Example:

```
function showMessage() {
    alert("Hello!");
}
```

It defines **how the webpage behaves and responds to user actions**.

In Simple Terms

HTML = Structure

CSS = Design

JavaScript = Behavior

Together, they create complete and interactive web pages.

Q10. Short Answer

Answer the following:

- a. What is the role of Bootstrap in web development?
- b. What is jQuery and why was it widely used in web development?
- c. What is React.js and what problem does it solve in frontend development?

Ans.

a. What is the role of Bootstrap in web development?

Bootstrap is a front-end CSS framework used to create **responsive and mobile-friendly websites** quickly. It provides pre-designed components such as buttons, forms, cards, navigation bars, and a responsive grid system.

b. What is jQuery and why was it widely used in web development?

jQuery is a **JavaScript library** that simplifies tasks such as DOM manipulation, event handling, animations, and AJAX requests. It was widely used because it provided a simple syntax and helped developers handle browser differences more easily.

c. What is React.js and what problem does it solve in frontend development?

React.js is a **JavaScript library for building user interfaces** using reusable components. It helps solve the problem of efficiently managing and updating complex, dynamic user interfaces by using a component-based approach and Virtual DOM.